



Python for Fixed Income

COURSE 3 · MARKET DATA AND TIME SERIES

Rolling Windows and Volatility

WEEK 2 · DAY 2

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

Five choices behind one number

Choice	This course
Rows or calendar days?	Rows: 21, 63, 252 (about 1, 3, 12 months)
Which rows?	Trailing: the window ends on today
Window not full yet?	No value: NaN until it is full
Sample or population?	Sample: ddof=1, Excel STDEV.S
Per day or per year?	Per day, unless the name says _annual

The desk's question

How much has the 10-year yield been moving lately? One number answers it: a volatility.

Volatility describes how much a series moved over its window. It says nothing about what comes next.

A window in rows against a window in days

`rolling(21)`

The last 21 rows. Every full window holds 21 changes.

`rolling("30D")`

Every row in 30 calendar days: from 18 to 22 rows, by weekends and holidays.

Rows in a 30-day window, 10 Yr daily changes

18 rows		5 dates
19 rows		106 dates
20 rows		606 dates
21 rows		1,528 dates
22 rows		674 dates

This course counts windows in rows: 21, 63, and 252, called about one month, three months, one year. Conventions, not calendar months.

Trailing, right-aligned, and min_periods

change_bp	mean_3	min_periods=1
1.0	NaN	1.00
3.0	NaN	2.00
2.0	2.00	2.00
5.0	3.33	3.33
4.0	3.67	3.67
6.0	5.00	5.00

A 3-row mean: each value is that row and the two before it. The partial column looks the same and is over fewer rows.

Excel: a relative range, filled down (tested)

=AVERAGE(B2:B4) in C4, filled down

C5 becomes =AVERAGE(B3:B5). C2 and C3 stay empty, like NaN.

=AVERAGE(OFFSET(B4,-2,0,3,1)) in D4

Two rows up, three rows tall: the same window. Both columns: 2.00, 3.33, 3.67, 5.00.

Full windows only: min_periods equals the window. A blank is honest.

rolling_vol_bp: volatility of daily changes in bp

```
def rolling_vol_bp(changes_bp, window=63, annualise=False):  
    """...sample standard deviation over a trailing window..."""  
    _check_series(changes_bp, "changes_bp")  
    vol = changes_bp.rolling(window, min_periods=window).std(ddof=1)  
    if annualise:  
        vol = vol * math.sqrt(TRADING_DAYS_PER_YEAR)  
    return vol
```

5.29 bp

21-row volatility of the
10 Yr on the last date,
per day

2.4e-14

bp: the largest gap from
Excel's STDEV.S on 190
dates

=STDEV.S(F3:F23) in G23, filled down

Always on daily changes, never on levels. The first change is NaN, so pass `change_bp(...).dropna()`.

20 NaN, then the first full window. It describes how much the yield moved, nothing more.

Sample against population

	Divides by	pandas	NumPy	Excel
Sample	$n - 1$	<code>.std()</code>	<code>np.std(x, ddof=1)</code>	STDEV.S
Population	n	<code>.std(ddof=0)</code>	<code>np.std(x)</code>	STDEV.P

5.294651 bp

sample, last 21 changes

5.167051 bp

population, the same 21

1.024695

ratio: $\sqrt{21 / 20}$

The trap

`np.std()` is population; `pandas .std()` is sample. Name `ddof`.

Excel: `=STDEV.P(F3:F23)` beside `STDEV.S`. `STDEV.S` of one value shows `#DIV/0!`.

Annualising: the factor, labelled

Market days in each full year

2015:	251	2021:	251
2016:	250	2022:	249
2017:	250	2023:	250
2018:	249	2024:	250
2019:	250	2025:	249
2020:	251		

5.29 bp

21-row volatility, per day

84.05 bp

the same, per year: $\times \sqrt{252}$

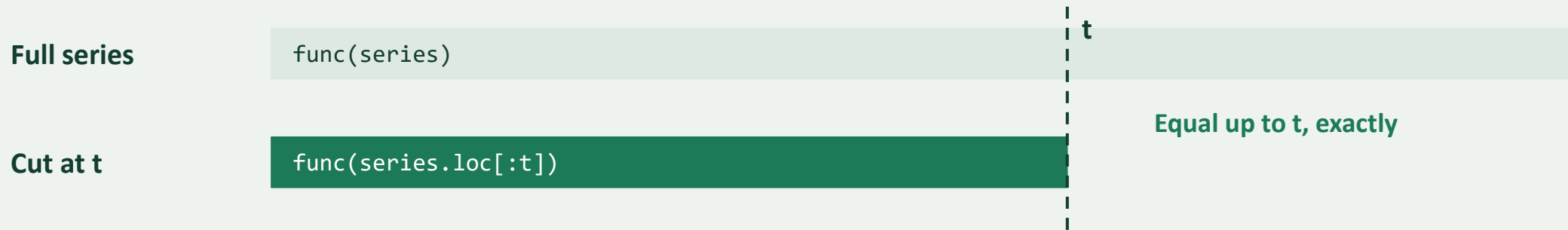
=G23*SQRT(252) in I23, filled down

Never 252 in this file. Other desks use other factors.

252 is a convention. Per day by default; per year only with `annualise=True` and a name ending `_annual`.

Lookahead and assert_no_lookahead

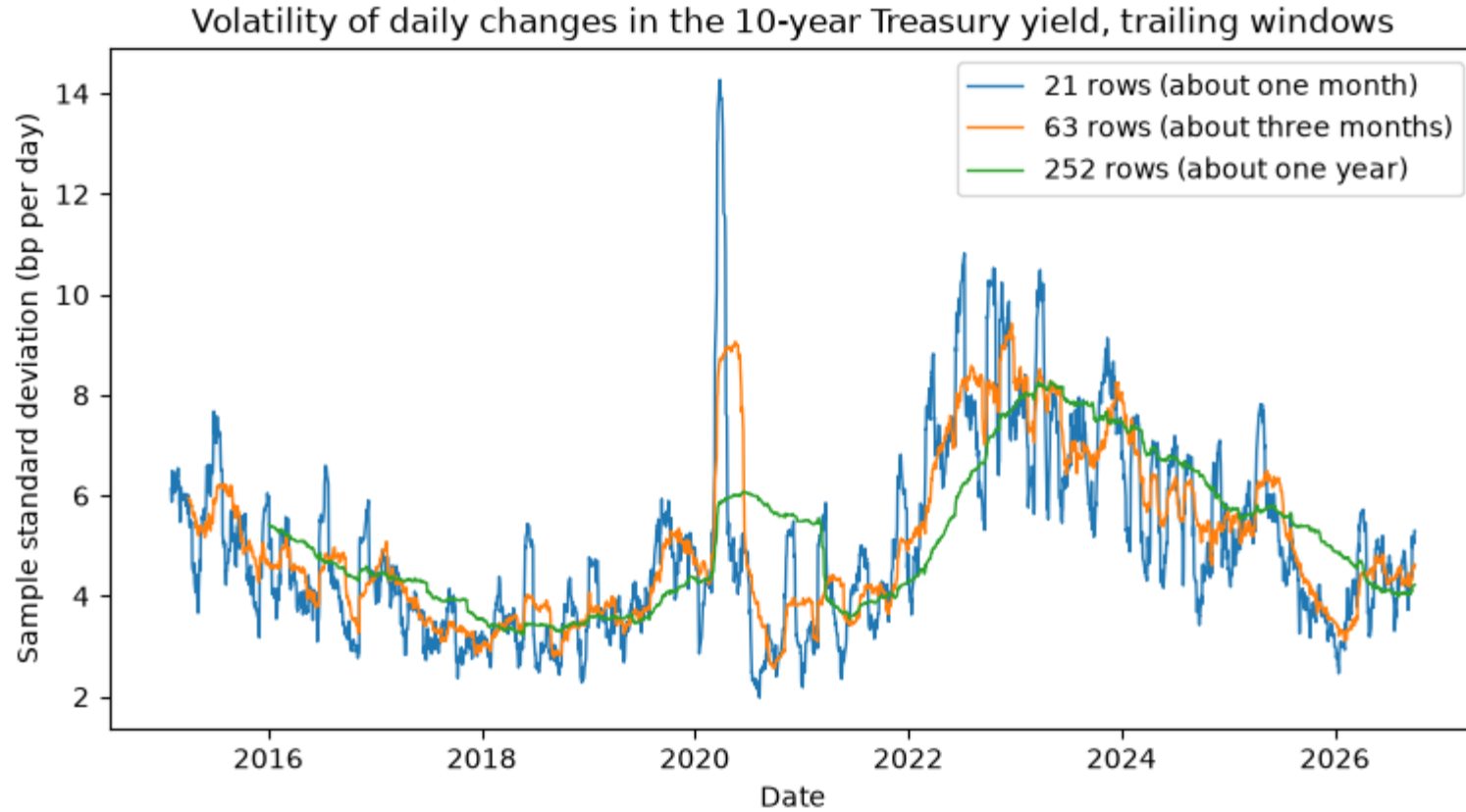
Lookahead: a number for date t that uses data from after t . The test: the output up to t is the same whether the series stops at t or runs on.



```
shift(-1): up to 2025-04-17, 7.8057 bp from the full series, nan from the cut one
```

From today every window function has a no-lookahead test. 29 tests pass.

21, 63, and 252 rows on one chart



bp per day, over the file

21 rows

low 1.98, high 14.27

on 2026-10-02: 5.29

63 rows

low 2.57, high 9.43

on 2026-10-02: 4.63

252 rows

low 3.23, high 8.26

on 2026-10-02: 4.22

Excel side by side

Excel side by side (tested in Excel)

`=STDEV.S(F3:F23) (G23)`

21-row volatility, sample

`=STDEV.P(F3:F23) (H23)`

population, beside it

`=G23*SQRT(252) (I23)`

per year, the factor in sight

`=STDEV.S(F3:F65) (J65)`

63 rows

`=STDEV.S(F3:F254) (K254)`

252 rows

`=AVERAGE(OFFSET(D64, -62,0,63,1)) (M64)`

63-row mean with OFFSET

Each written once in the first row with a full window, filled down over 2,940 rows, and compared with Python on every row: gaps below 1e-12 bp.

The window is the range. Where the first formula goes is min_periods.

Save the day: yesterday's file, and back

1. Morning

Commit the start-of-day files

2. Evening

Commit today's two files

3. Look back

restore --source=HEAD~1: 389
lines

4. Come back

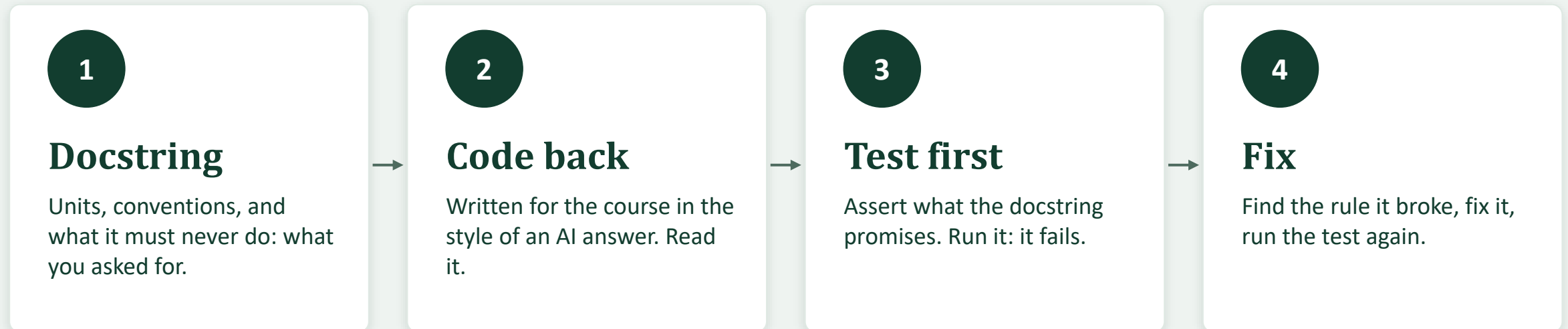
restore: 417 lines again

```
git -C "{work}" restore --source=HEAD~1 marketdata.py # yesterday's file in the folder
git -C "{work}" restore marketdata.py                 # back to the last commit
```

The commits never change; only the file in the folder does. Check git status before a plain restore.

The AI check

The docstring is the prompt. The code that comes back runs, and has one data bug.



Today: `trailing_vol_bp` uses `rolling(63, center=True)`: each value uses the 31 rows after its date.

Code that runs is not the same as code that is right.

What you can do now

- Count windows in rows, trailing, full windows only
- `rolling_vol_bp`: sample standard deviation of daily changes in bp
- Sample against population: `STDEV.S`, `STDEV.P`, and the NumPy default
- Annualise by `sqrt(252)` only when labelled: a convention
- Prove no future data with `assert_no_lookahead`

Week 2, Day 3 Z-scores and percentile ranks: where today sits in its own history